

BITS Pilani – M.Tech Software Engineering (Sem 3)

Course: SS ZG566 – Secure Software Engineering

Mid-Semester Test – Solutions

Q1: Brief responses (50 words each)

a) Define malware. How does a worm differ from a virus?

Malware is malicious software designed to disrupt, damage, or gain unauthorized access to systems. A worm is self-replicating and spreads without user intervention across networks, while a virus requires user action (like running an infected file) to activate and propagate.

b) Do we mean the same by terms Vulnerability and Threat?

No. A vulnerability is a weakness in software or system that can be exploited, while a threat is the potential danger or actor that exploits the vulnerability. Example: SQL injection flaw (vulnerability) vs. hacker exploiting it (threat).

c) Is it a good idea to have laser-like focus on (a specific) threat during threat modeling?

No. Focusing only on one threat narrows the defense scope and may leave systems exposed to other attack vectors. Threat modeling should consider a wide range of possible threats, ensuring holistic coverage across attack surfaces.

d) Characterize Every-sprint, Bucket, and One-Time Requirements of Microsoft SDL-Agile with examples.

Every-sprint: Applied continuously (e.g., code review, static analysis). Bucket: Addressed periodically (e.g., pen testing every few sprints). One-time: Applied once per project (e.g., threat model creation). This ensures balanced security practices across the lifecycle.

e) Briefly differentiate security requirements from privacy requirements.

Security requirements protect data and systems from unauthorized access, tampering, or disruption (e.g., encryption, authentication). Privacy requirements govern how personal data is collected, stored, and shared, ensuring compliance with laws like GDPR (e.g., user consent, data minimization).

f) How can you utilize MITRE CWEs during software development?

MITRE CWE (Common Weakness Enumeration) catalog provides a list of known software weaknesses. Developers can use it for secure coding checklists, guiding code reviews, designing automated static analysis rules, and prioritizing fixes for high-risk vulnerabilities.

Q2: CMU SQUARE vs. Requirements Gathering (100 words)

Traditional software engineering requirements gathering emphasizes functional and non-functional goals for usability, performance, and features. In contrast, CMU's SQUARE (Security Quality Requirements Engineering) process integrates security considerations early. SQUARE involves steps like agreeing on definitions, identifying security goals, risk assessment, eliciting security requirements, and prioritization. Unlike generic requirement elicitation, SQUARE explicitly incorporates threat modeling, security categories, and stakeholder collaboration to ensure security requirements are measurable and testable. This structured approach prevents afterthought security fixes, making systems resilient against attacks. Thus, SQUARE augments traditional processes by embedding a security-first mindset throughout requirements engineering.

Q3: Secure Logger Pattern & Cyber Kill Chain (100 words)

The Secure Logger pattern ensures that security-relevant events (logins, privilege changes, data access) are recorded reliably, tamper-proof, and confidentially. Logs are critical for auditing, forensics, and intrusion detection. Within the cyber kill chain, secure logging aids in detection and response phases—identifying anomalies, tracking attacker activities, and enabling timely countermeasures. For example, abnormal repeated failed logins can signal brute-force attempts.

Unlike traditional logging, secure logging applies encryption, access controls, and integrity checks to prevent adversaries from erasing traces. Hence, it strengthens resilience by ensuring accurate visibility into system activities during and after cyberattacks.

Q4: Open-Source Considerations – Log4j Case (100 words)

The Log4j vulnerability highlighted risks in open-source dependencies. Considerations include: 1) Dependency visibility – maintain SBOM (Software Bill of Materials) to know all components used. 2) Patch management – continuously monitor for updates and apply patches quickly. 3) Minimalism – avoid unnecessary libraries to reduce attack surface. 4) Trust and governance – use components from active, reputable communities. 5) Security testing – regularly scan dependencies with tools like OWASP Dependency-Check. 6) Isolation – sandbox or containerize to limit damage. Lessons show that blindly trusting open-source can expose systems unless supported by proactive risk management.

Q5: Automation in Security Testing (4 Marks)

Static Application Security Testing (SAST): Highly automatable—code scans detect vulnerabilities early, integrated in CI/CD. Dynamic Application Security Testing (DAST): Automated tools simulate runtime attacks, but may miss logic flaws. Interactive Application Security Testing (IAST): Automation effective when integrated with runtime monitoring. Penetration Testing: Limited automation; requires human expertise for business logic attacks. Fuzz Testing: Effective automation for input mutation and crash discovery. Dependency Scanning: Fully automatable via tools like Snyk. Automation increases speed, coverage, and consistency, but human oversight remains crucial for contextual and logic-based issues.